

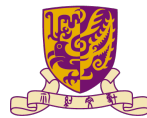
Network Programming

Routing

Minchen Yu

SDS@CUHK-SZ

Spring 2026



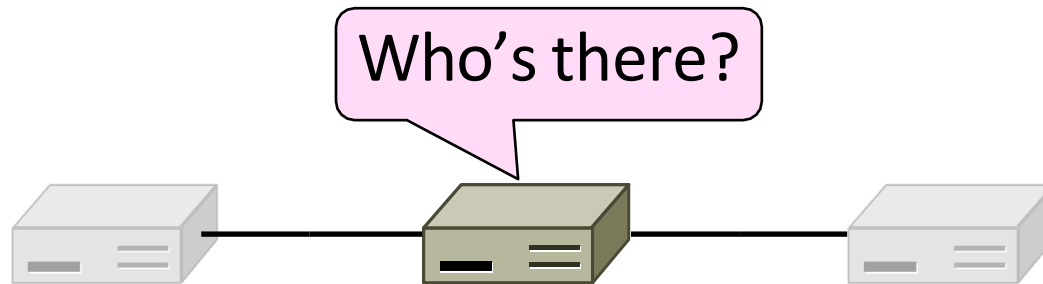
香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen



SCHOOL OF
DATA SCIENCE
數據科學學院

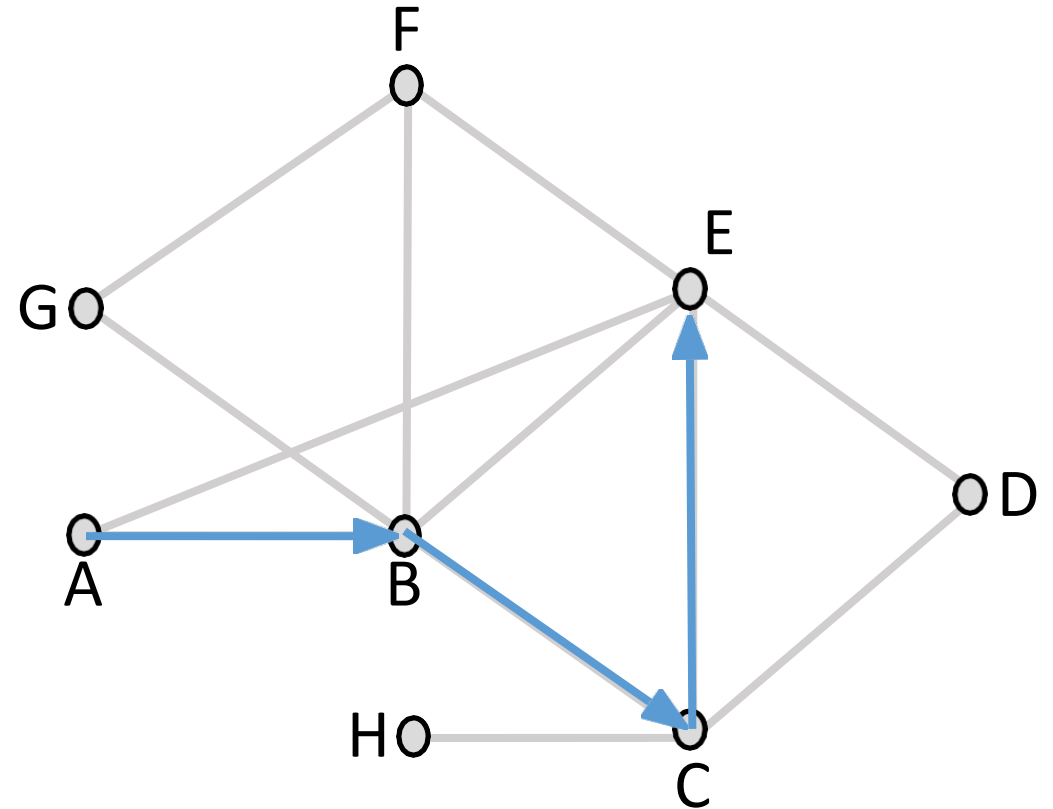
Recall: Rules of Distributed Routing

- All nodes are alike; no controller
- Nodes learn by exchanging messages with neighbors
- Nodes operate concurrently
- There may be node/link/message failures



What are “Best” paths anyhow?

- Many possibilities:
 - Latency, avoid circuitous paths
 - Bandwidth, avoid slow links
 - Money, avoid expensive links
 - Hops, to reduce switching



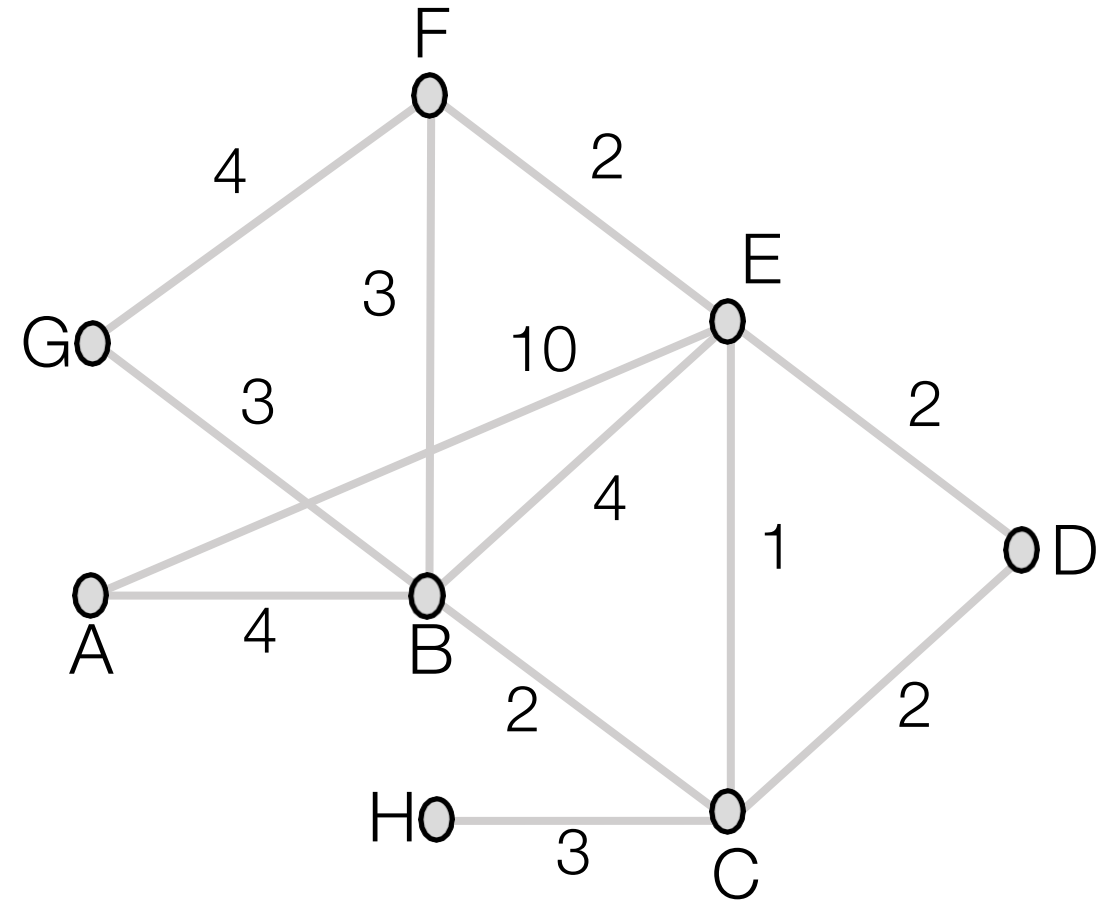
Shortest Paths

We'll approximate “best” by a cost function that captures the factors

- Often call lowest “shortest”
- Assign each link a cost (distance)
- Define best path between each pair of nodes as the path that has the lowest total cost (or is shortest)
- Pick randomly to any break ties

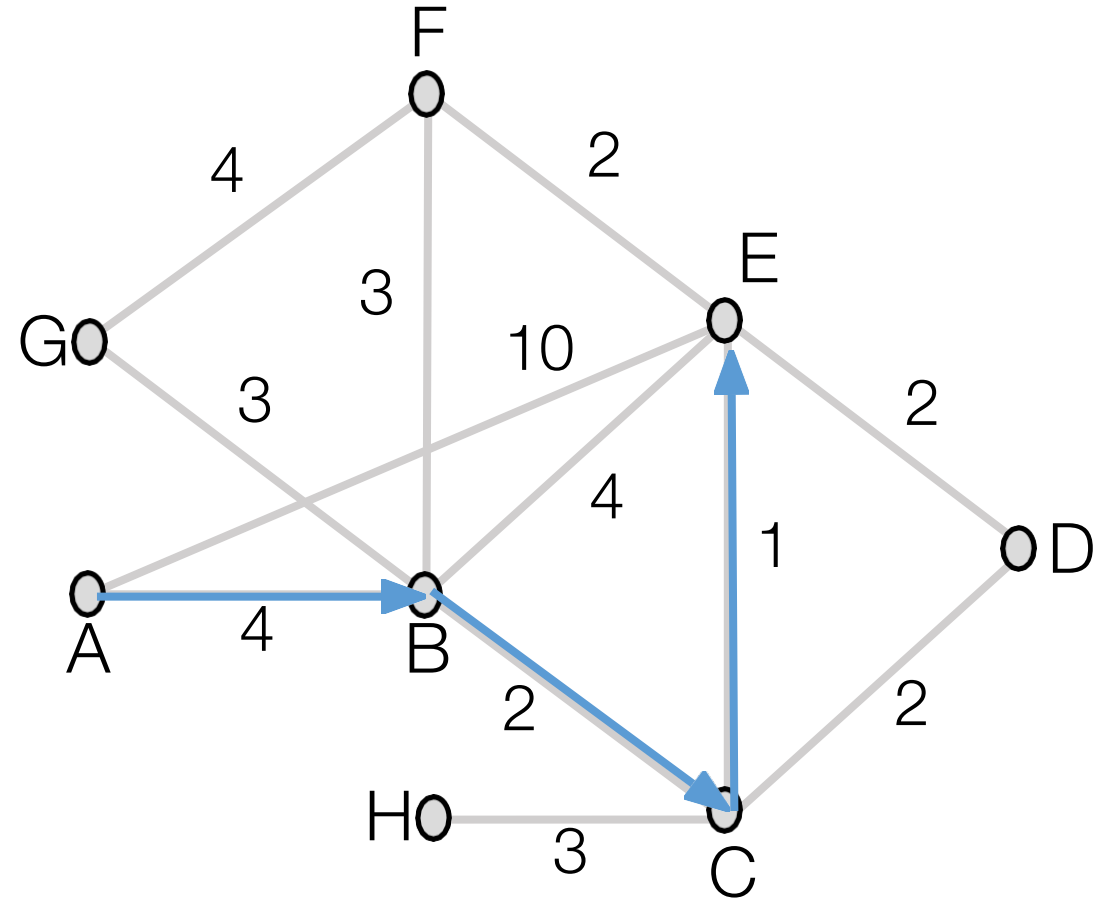
Shortest Paths (2)

- Find the shortest path $A \rightarrow E$
- All links are bidirectional, with equal costs in each direction
 - Can extend model to unequal costs if needed



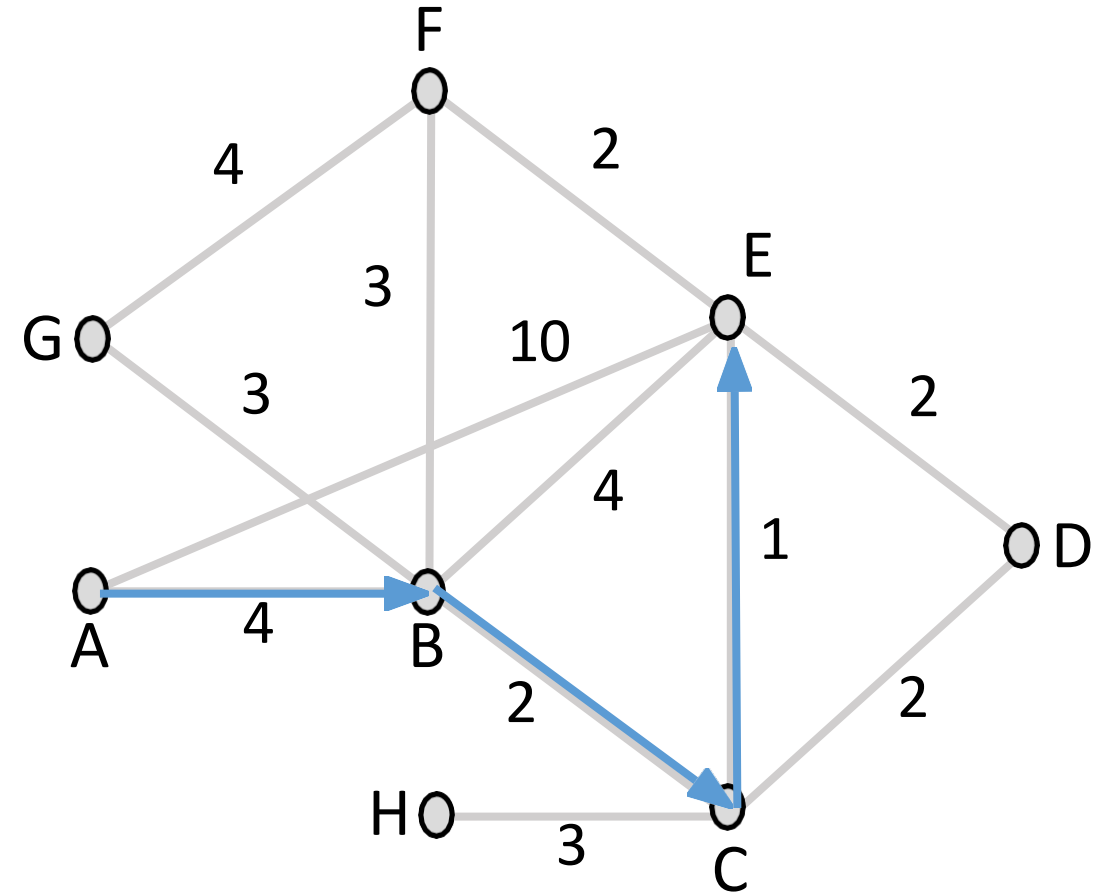
Shortest Paths (3)

- ABCE is a shortest path
- $\text{dist}(\text{ABCE}) = 4 + 2 + 1 = 7$
- This is less than:
 - $\text{dist}(\text{ABE}) = 8$
 - $\text{dist}(\text{ABFE}) = 9$
 - $\text{dist}(\text{AE}) = 10$
 - $\text{dist}(\text{ABCDE}) = 10$



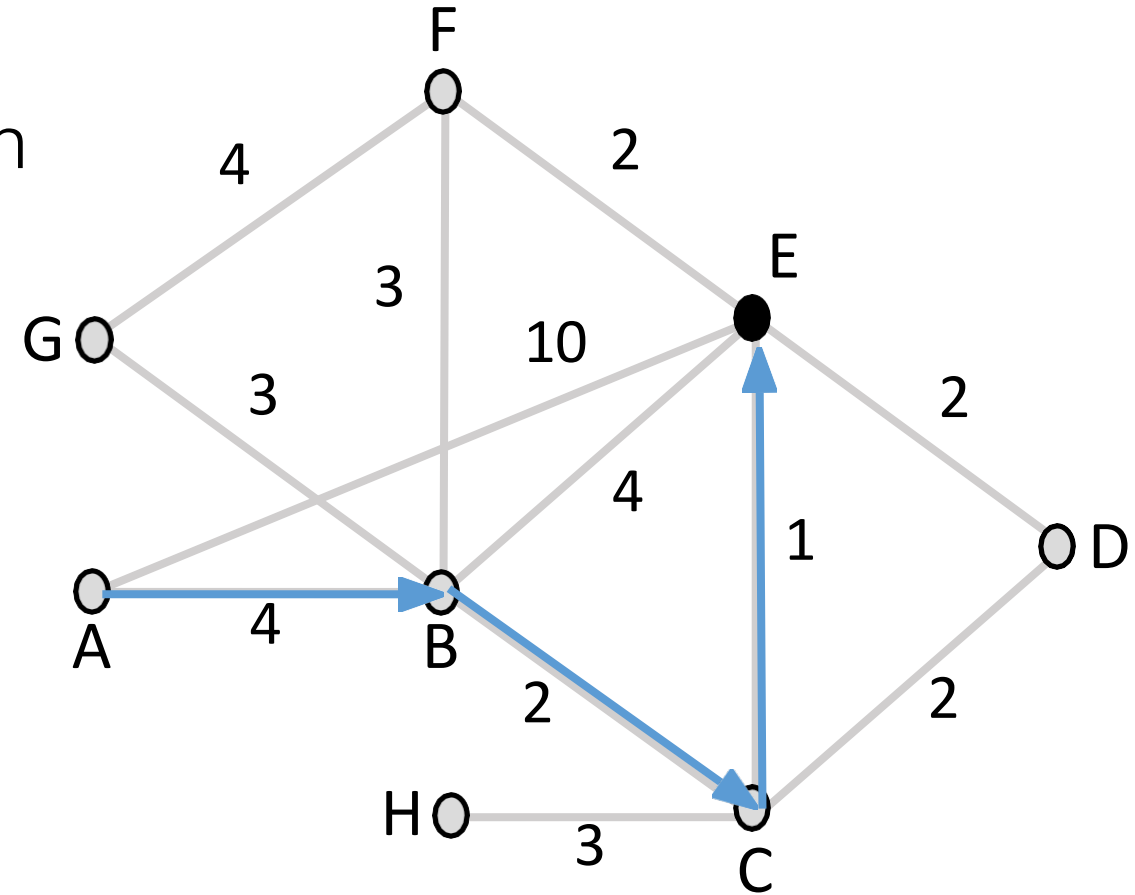
Shortest Paths (4)

- Optimality property:
 - Subpaths of shortest paths are also shortest paths
- ABCE is a shortest path
 - So are ABC, AB, BCE, BC, CE



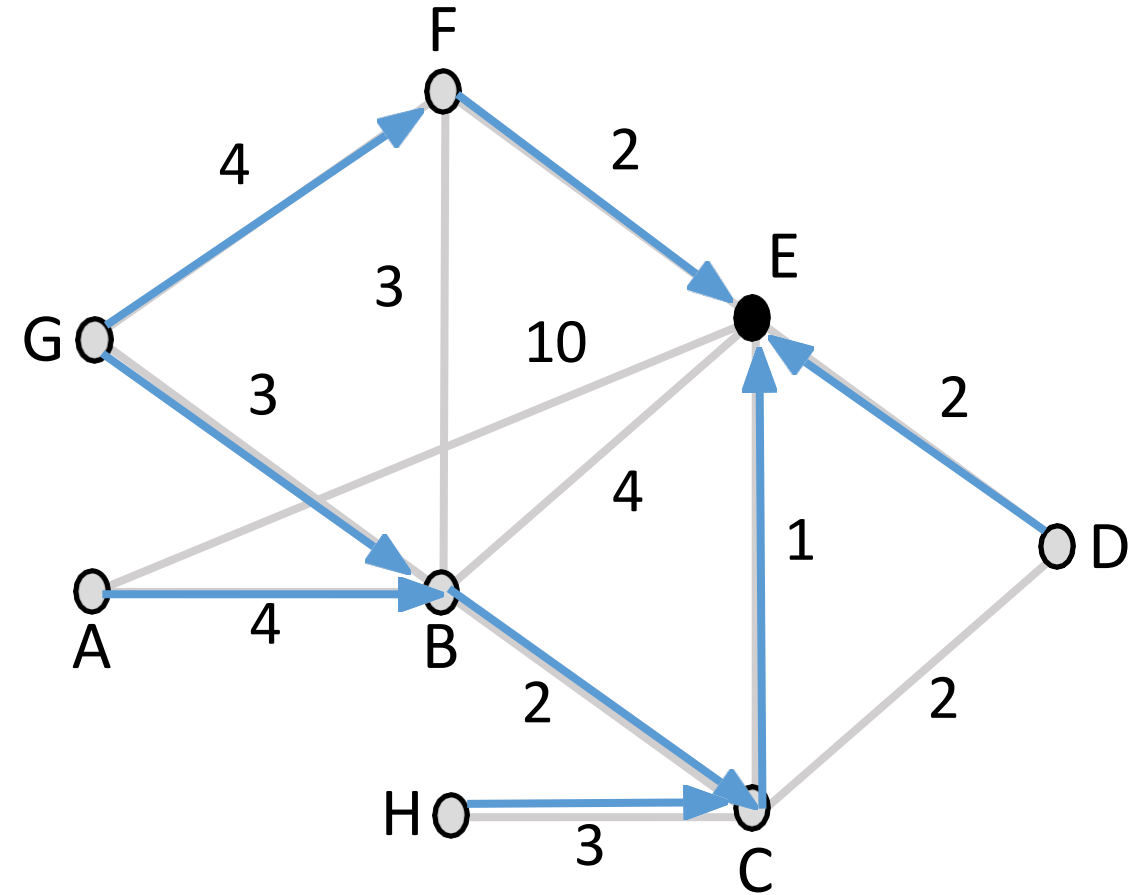
Sink Trees

- Sink tree for a destination is the union of all shortest paths towards the destination
 - Similarly source tree
- Find the sink tree for E



Sink Trees (2)

- Implications:
 - Only need to use destination to follow shortest paths
 - Each node only need to send to the next hop
- Forwarding table at a node
 - Lists next hop for each destination



Distance Vector Routing

Distance Vector Setting

Each node computes its forwarding table in a distributed setting:

1. Nodes know only the cost to their neighbors; not topology
2. Nodes can talk only to their neighbors using messages
3. All nodes run the same algorithm concurrently
4. Nodes and links may fail, messages may be lost

Distance Vector Algorithm

Each node maintains a vector of distances(and next hops) to all destinations

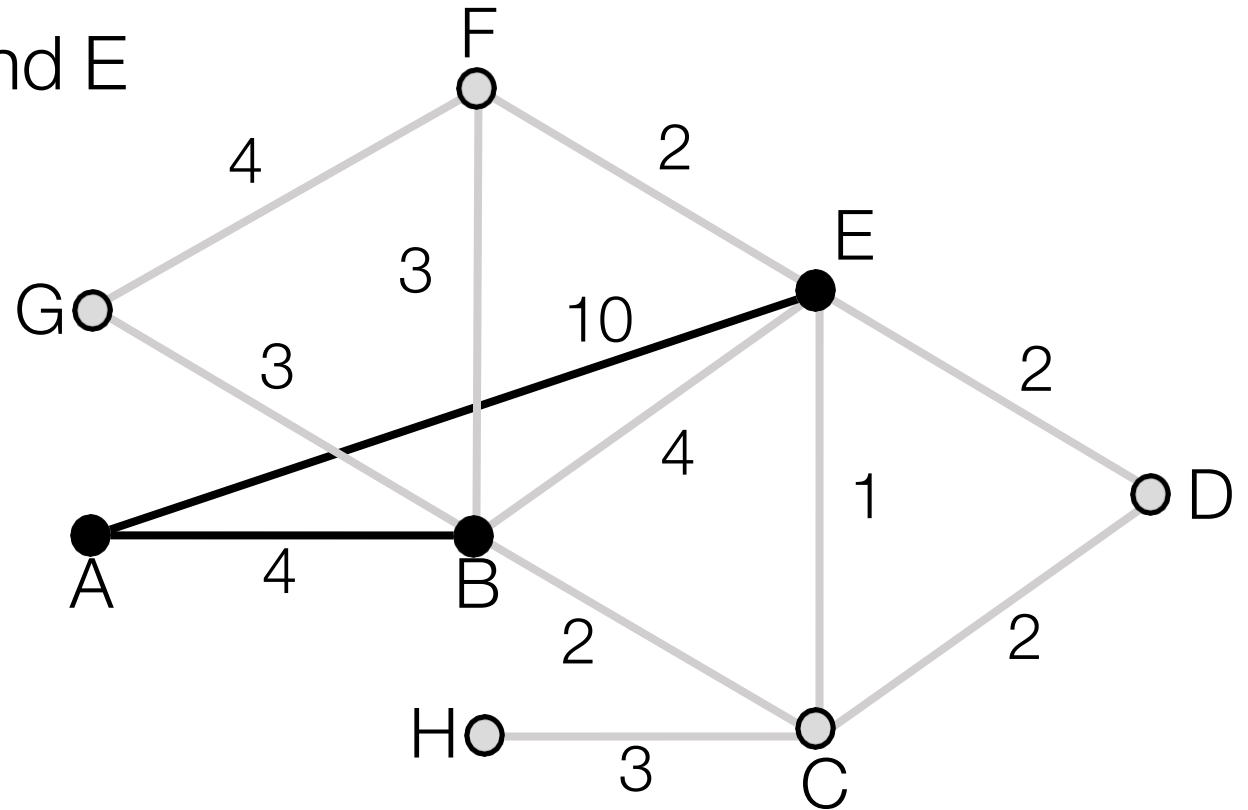
1. Initialize vector with 0 (zero) cost to self, ∞ (infinity) to other destinations
2. Periodically send vector to neighbors
3. Update vector for each destination by selecting the shortest distance heard, after adding cost of neighbor link
4. Use the best neighbor for forwarding

Distance Vector (2)

- Consider from the point of view of node A
 - Can only talk to nodes B and E

Initial
vector

To	Cost
A	0
B	∞
C	∞
D	∞
E	∞
F	∞
G	∞
H	∞

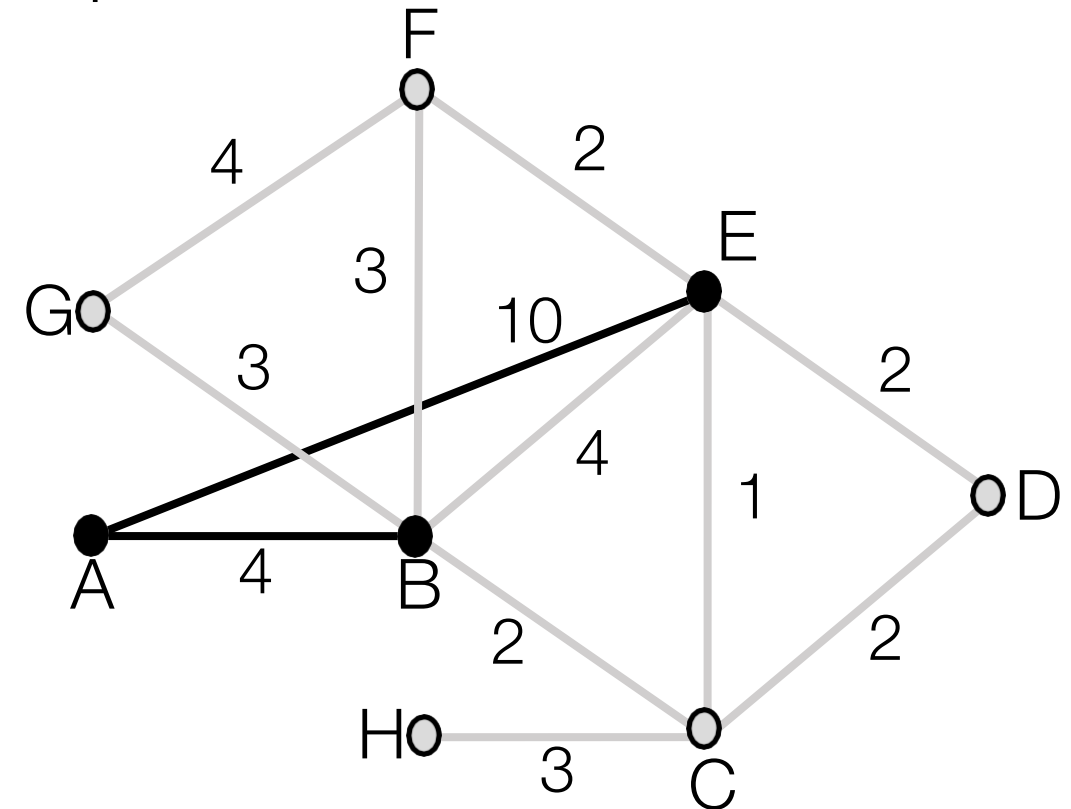


Distance Vector (3)

- First exchange with B, E; learn best 1-hop routes

To	B's	E's		B +4	E +10		Co st	Nex t
A	∞	∞		∞	∞		0	--
B	0	∞		4	∞		4	B
C	∞	∞	→	∞	∞		∞	--
D	∞	∞		∞	∞		∞	--
E	∞	0		∞	10		10	E
F	∞	∞		∞	∞		∞	--
G	∞	∞		∞	∞		∞	--
H	∞	∞		∞	∞		∞	--

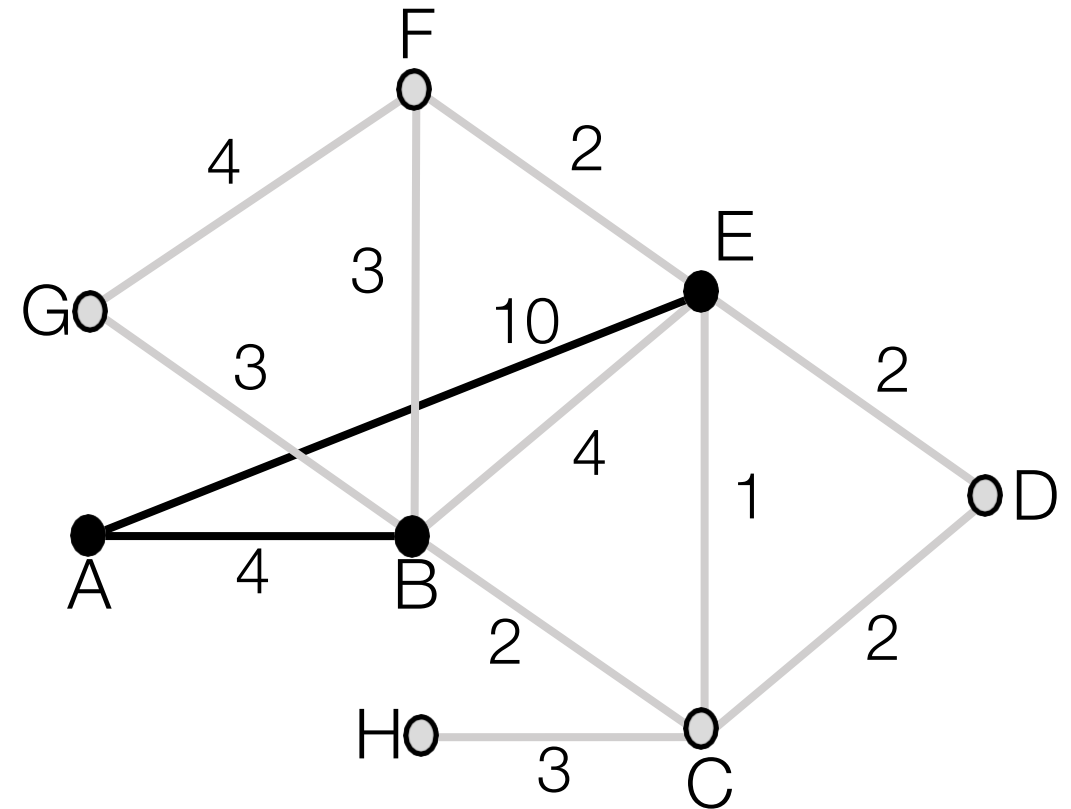
Learned better route								
----------------------	--	--	--	--	--	--	--	--



Distance Vector (4)

- Second exchange; learn best 2-hop routes

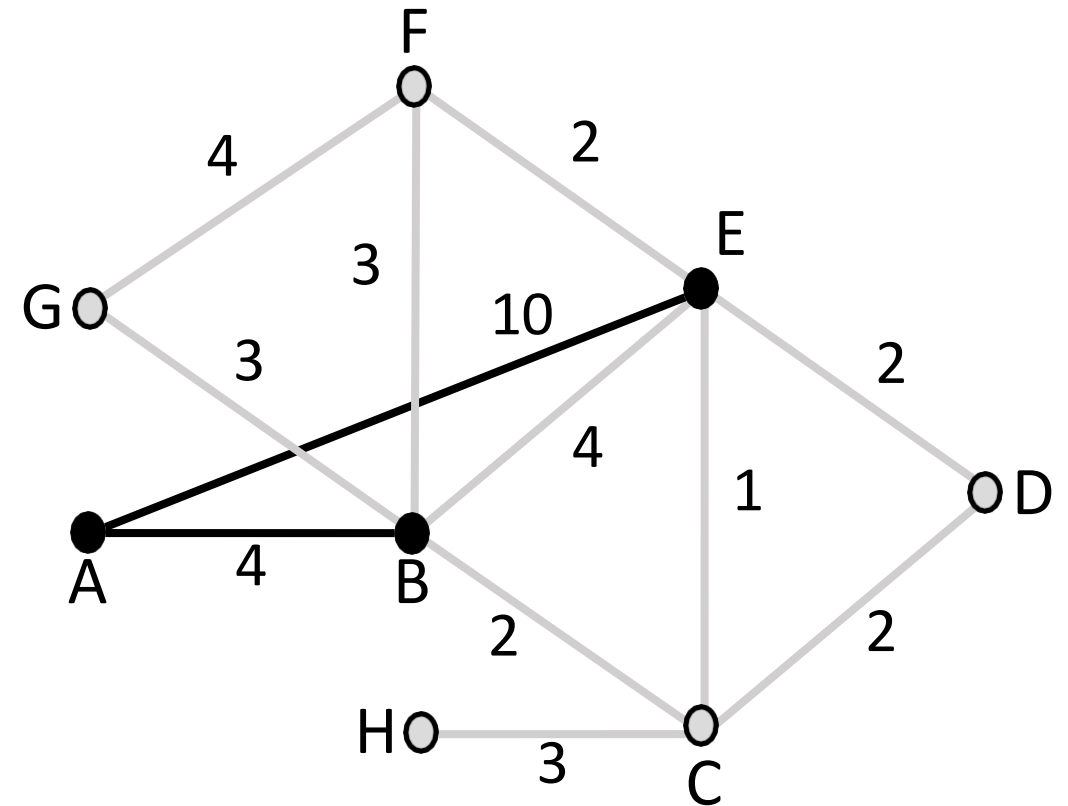
To	B's	E's	→	B +4	E +10	→	Co st	Nex t
A	4	10		8	20		0	--
B	0	4		4	14		4	B
C	2	1		6	11		6	B
D	∞	2		∞	12		12	E
E	4	0		8	10		8	B
F	3	2		7	12		7	B
G	3	∞		7	∞		7	B
H	∞	∞		∞	∞		∞	--



Distance Vector (5)

- Third exchange; learn best 3-hop routes

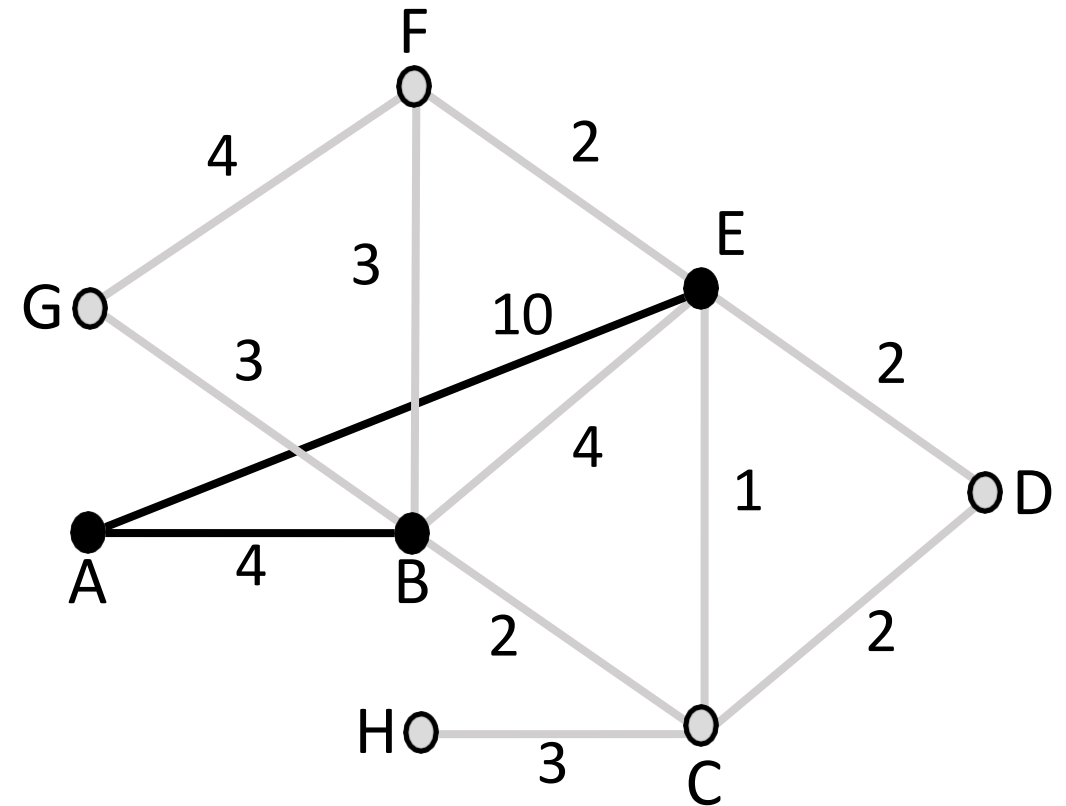
To	B's	E's	→	B +4	E +10	→	Co st	Nex t
A	4	8		8	18		0	--
B	0	3		4	13		4	B
C	2	1		6	11		6	B
D	4	2		8	12		8	B
E	3	0		7	10		7	B
F	3	2		7	12		7	B
G	3	6		7	16		7	B
H	5	4		9	14		9	B



Distance Vector (6)

- Subsequent exchanges; converged

To	B's	E's	→	B	E	→	Co	Nex
				+4	+10		st	t
A	4	7		8	17		0	--
B	0	3		4	13		4	B
C	2	1		6	11		6	B
D	4	2		8	12		8	B
E	3	0		7	10		7	B
F	3	2		7	12		7	B
G	3	6		7	16		7	B
H	5	4		9	14		9	B



Distance Vector Dynamics

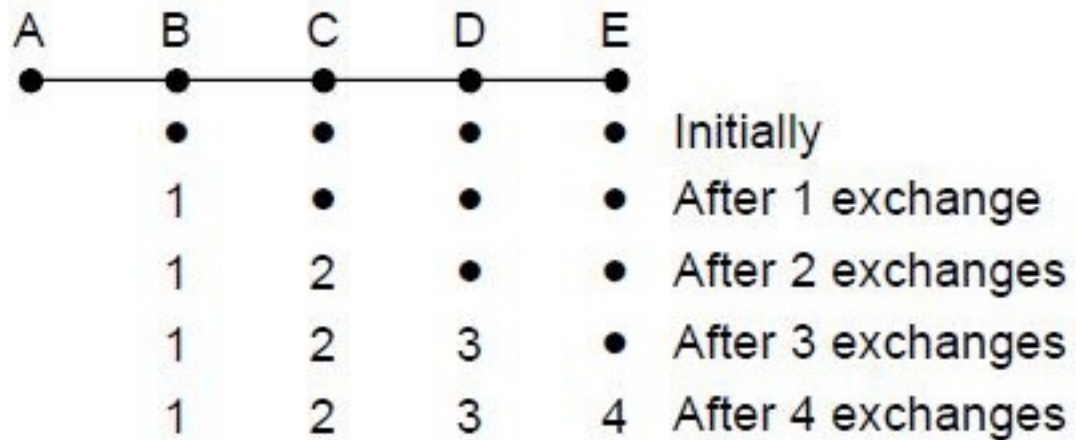
- Adding routes:
 - News travels one hop per exchange
- Removing routes:
 - When a node fails, no more exchanges, other nodes forget

Distance Vector Dynamics

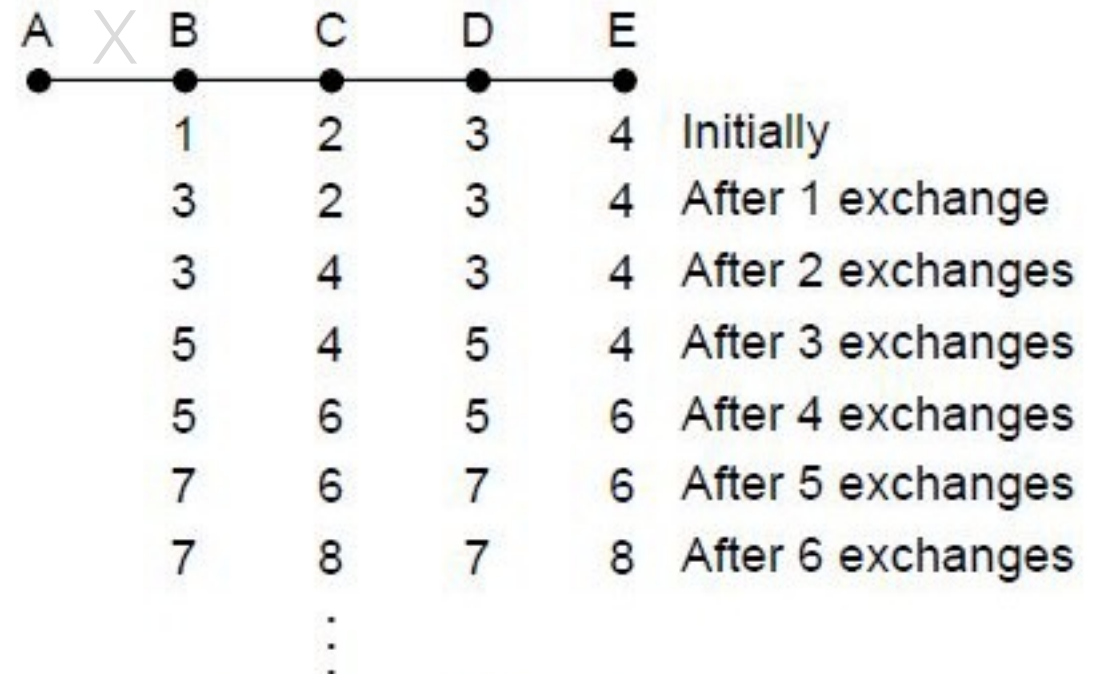
- Adding routes:
 - News travels one hop per exchange
 - Removing routes:
 - When a node fails, no more exchanges, other nodes forget
- Problem(s)?

DV Dynamics (2)

- Good news travels quickly, bad news slowly



Desired convergence



"Count to infinity" scenario

DV Dynamics (3)

- Various heuristics to address
 - “Split horizon”
 - Don’t send route back to where you learned it from.
 - Poison reverse
 - Send “infinity” when you notice a disconnect
- But none are very effective
 - Link state now favored in practice
 - Except when very resource-limited

RIP (Routing Information Protocol)

- DV protocol with hop count as metric
 - Infinity is 16 hops; limits network size
 - Includes split horizon, poison reverse
- Routers send vectors every 30 seconds
 - Runs on top of UDP
 - Time-out in 180 secs to detect failures
- RIPv1 specified in RFC1058 (1988)

Link-State Routing

Link-State Routing

- Other broad class of routing algorithms
 - Trades more computation than distance vector for better dynamics
- Widely used in practice
 - Used in Internet/ARPANET from 1979

Link-State Setting

Nodes compute their forwarding table in the same distributed setting as for distance vector:

1. Nodes know only the cost to their neighbors; not topology
2. Nodes can talk only to their neighbors using messages
3. All nodes run the same algorithm concurrently
4. Nodes/links may fail, messages may be lost

Link-State Algorithm

Proceeds in two phases:

1. Nodes flood topology with link state packets
 - Each node learns full topology
2. Each node computes its own forwarding table
 - By running Dijkstra's Algorithms (or equivalent)

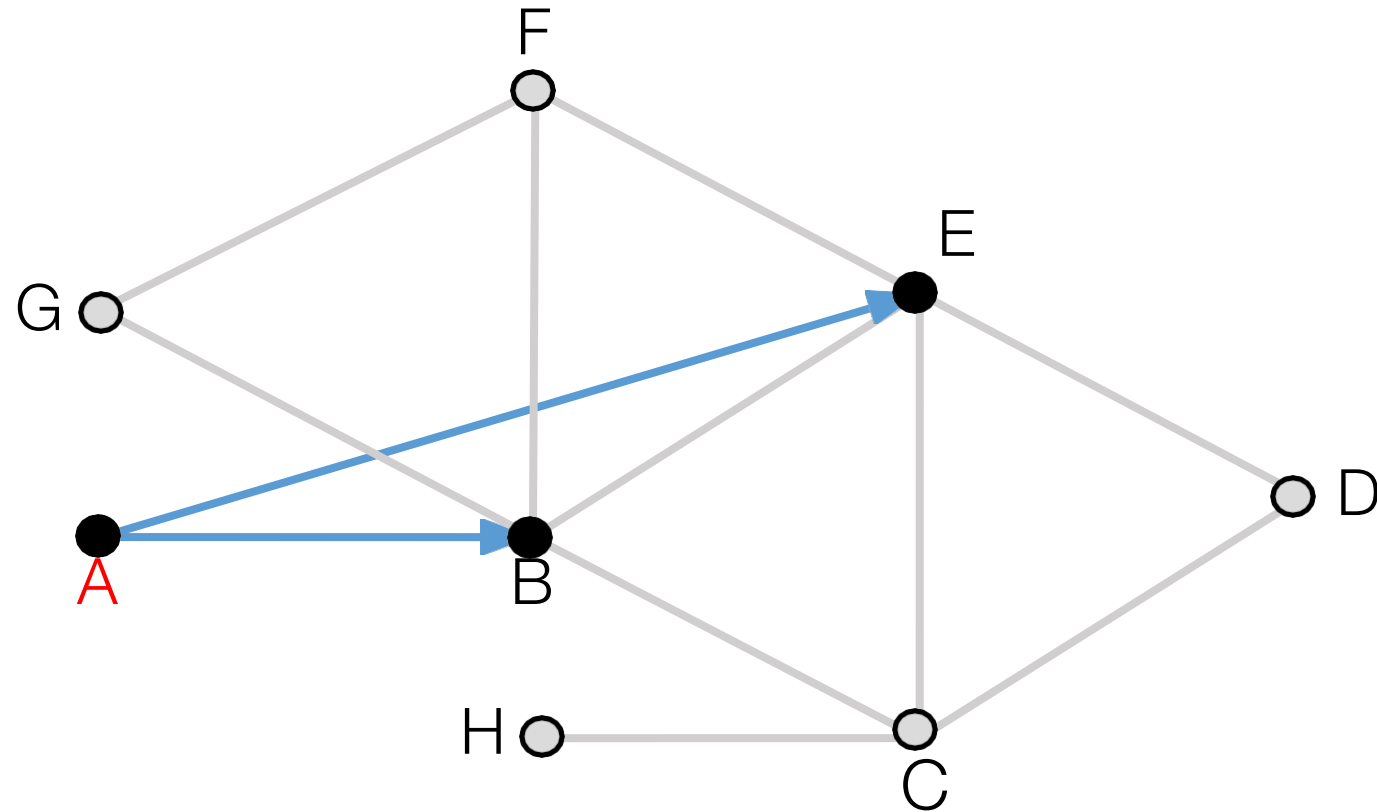
Link-State Part 1: Flood Routing

Flooding

- Rule used at each node:
 - Sends an incoming message on to all other neighbors
 - Remember the message so that it is only flooded once

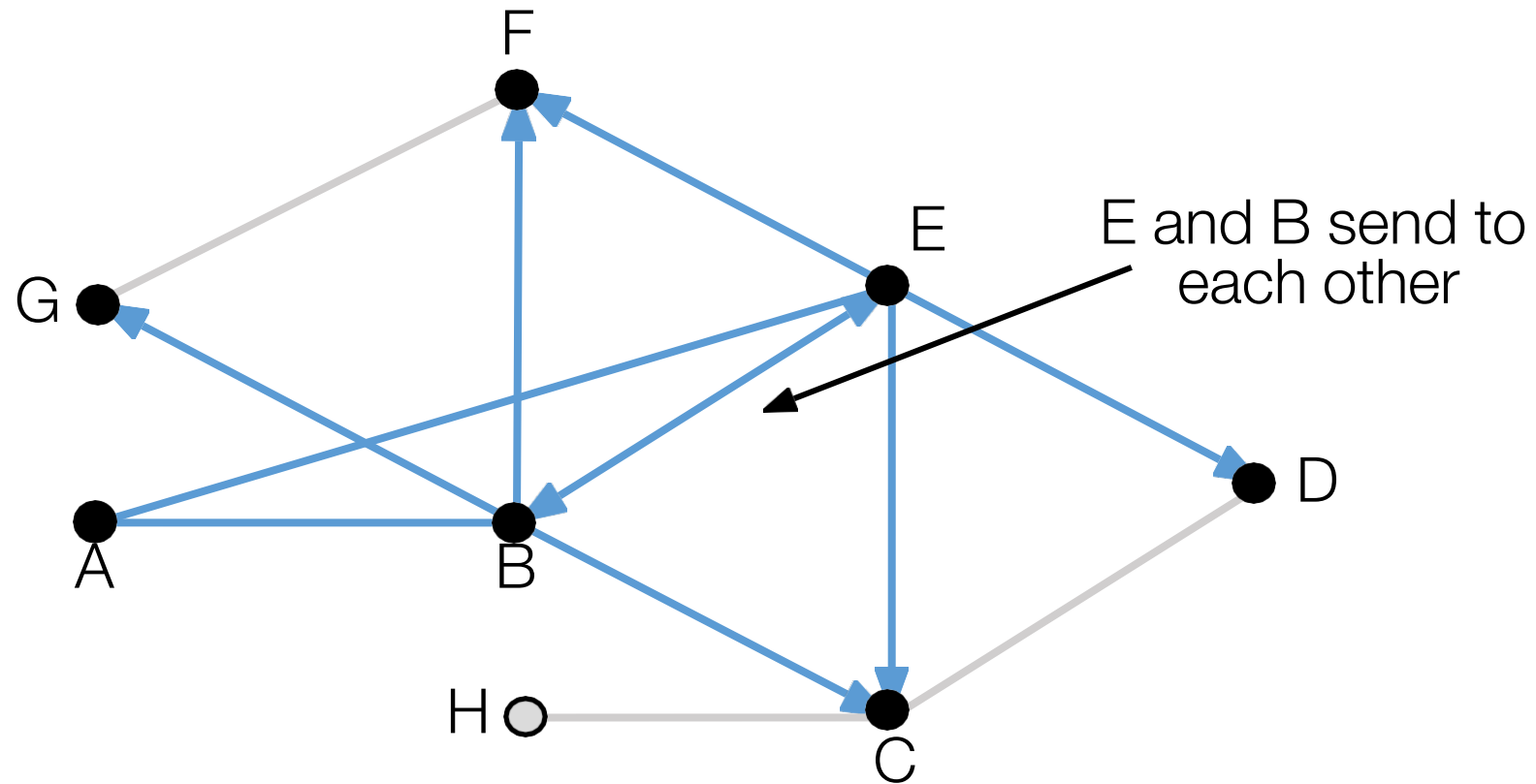
Flooding (2)

- Consider a flood from A; first reaches B via AB, E via AE



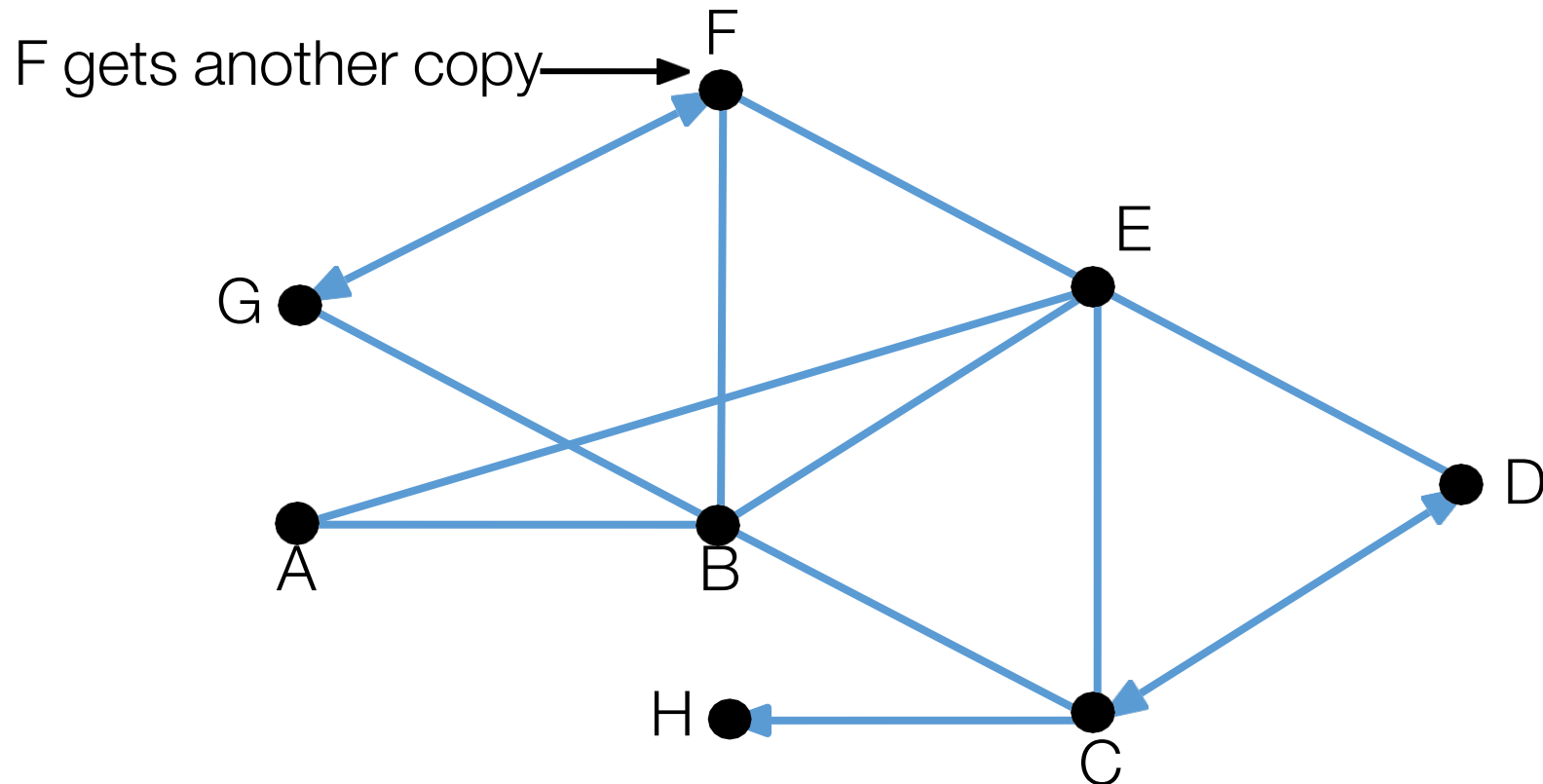
Flooding (3)

- Next B floods BC, BE, BF, BG, and E floods EB, EC, ED, EF



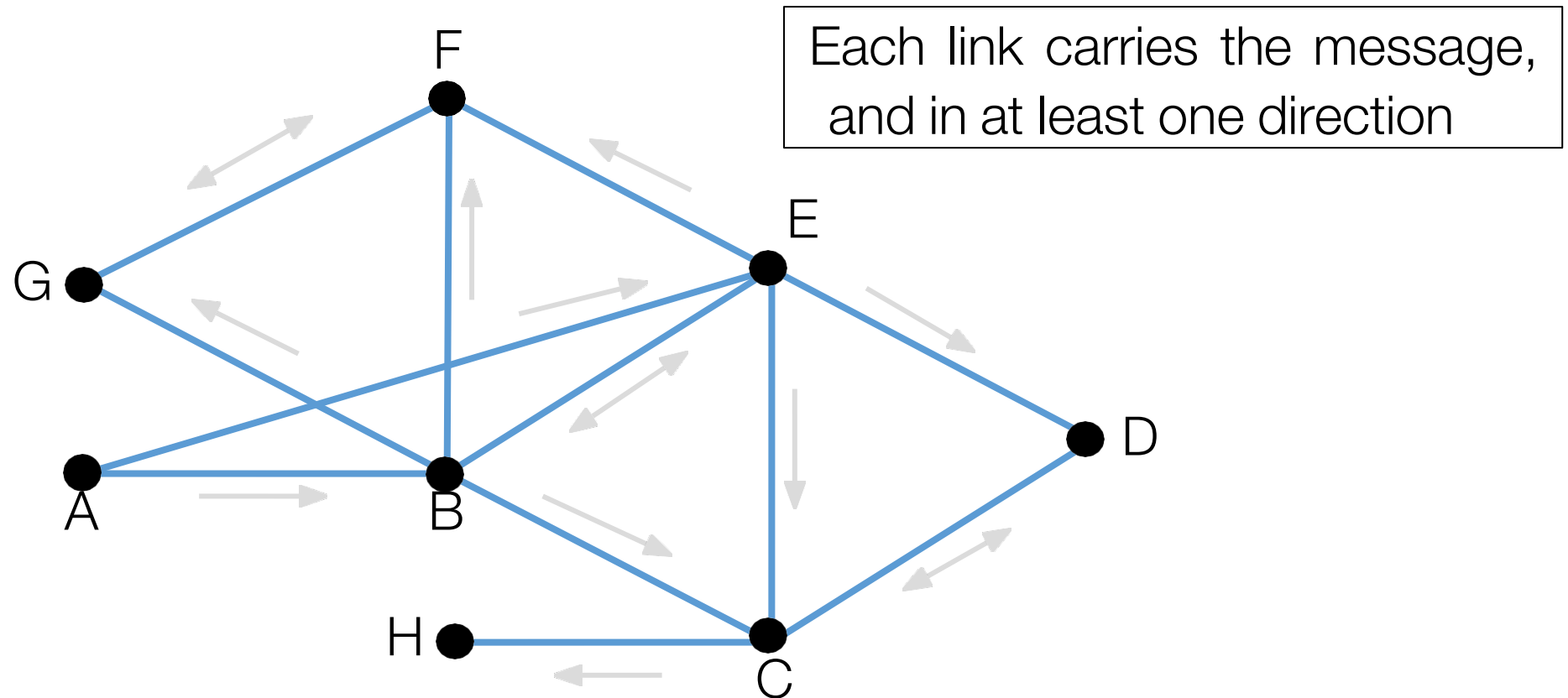
Flooding (4)

- C floods CD, CH; D floods DC; F floods FG; G floods GF



Flooding (5)

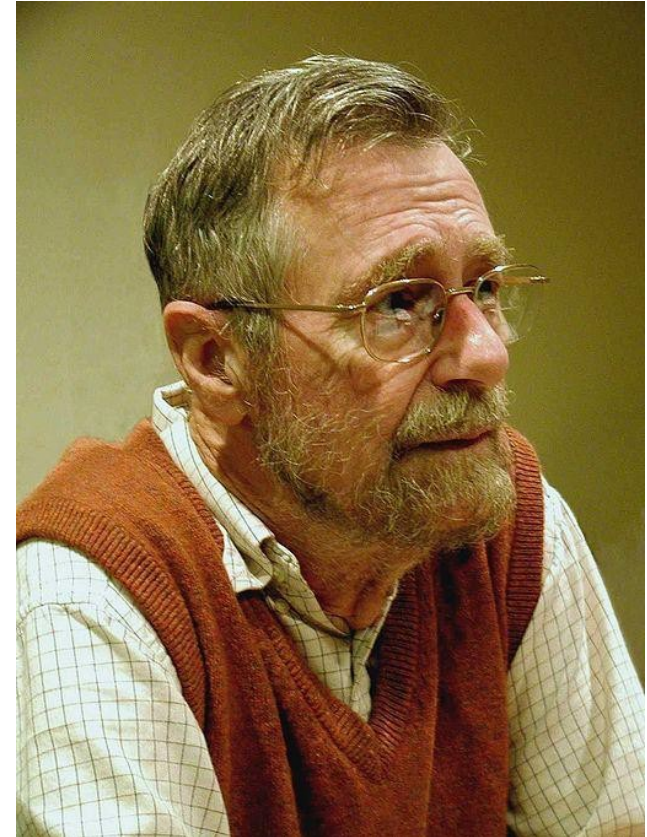
- H has no-one to flood ... and we're done



Link-State Part 2: Dijkstra's Algorithm

Edsger W. Dijkstra (1930-2002)

- Famous computer scientist
 - Programming languages
 - Distributed algorithms
 - Program verification
- Dijkstra's algorithm, 1969
 - Single-source shortest paths, given network with non-negative link costs



By Hamilton Richards, CC-BY-SA-3.0, via Wikimedia Commons

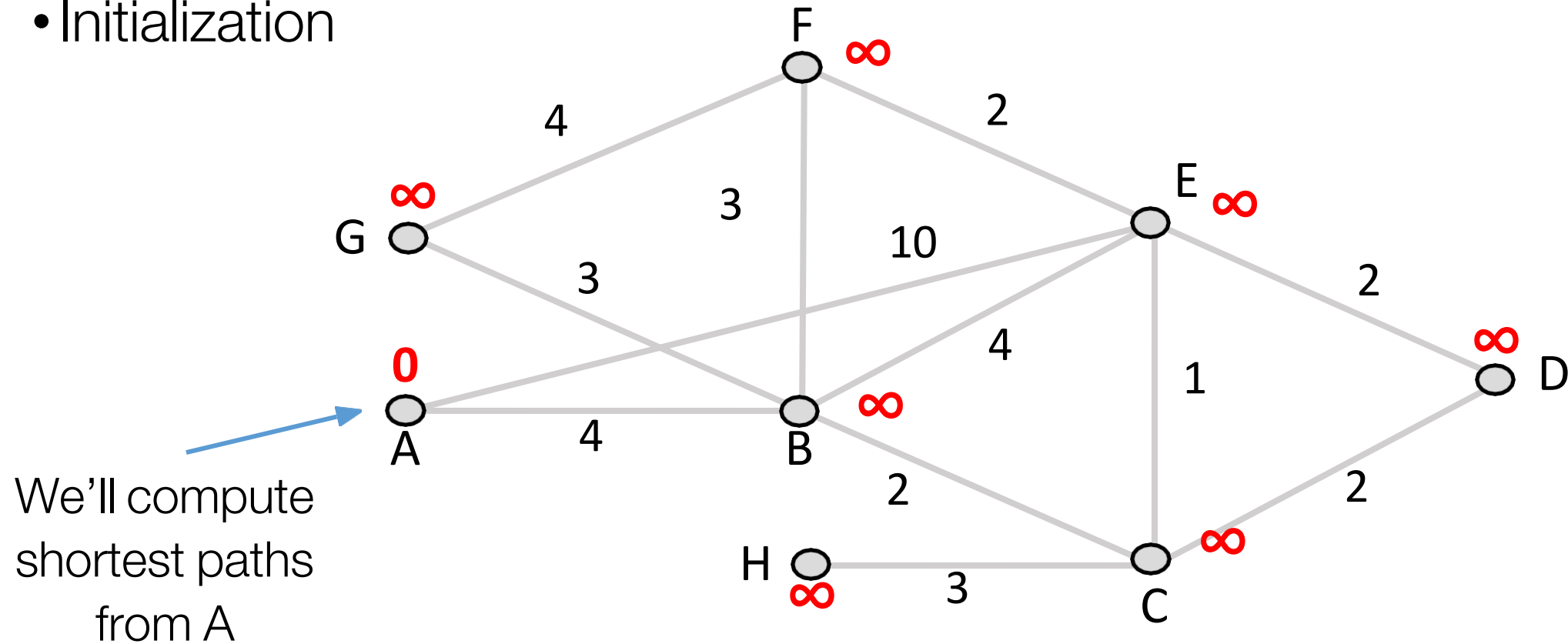
Dijkstra's Algorithm

Algorithm:

- Mark all nodes tentative, set distances from source to 0 (zero) for source, and ∞ (infinity) for all other nodes
- While tentative nodes remain:
 - Extract N, a node with lowest distance
 - Add link to N to the shortest path tree
 - Relax the distances of neighbors of N by lowering any better distance estimates

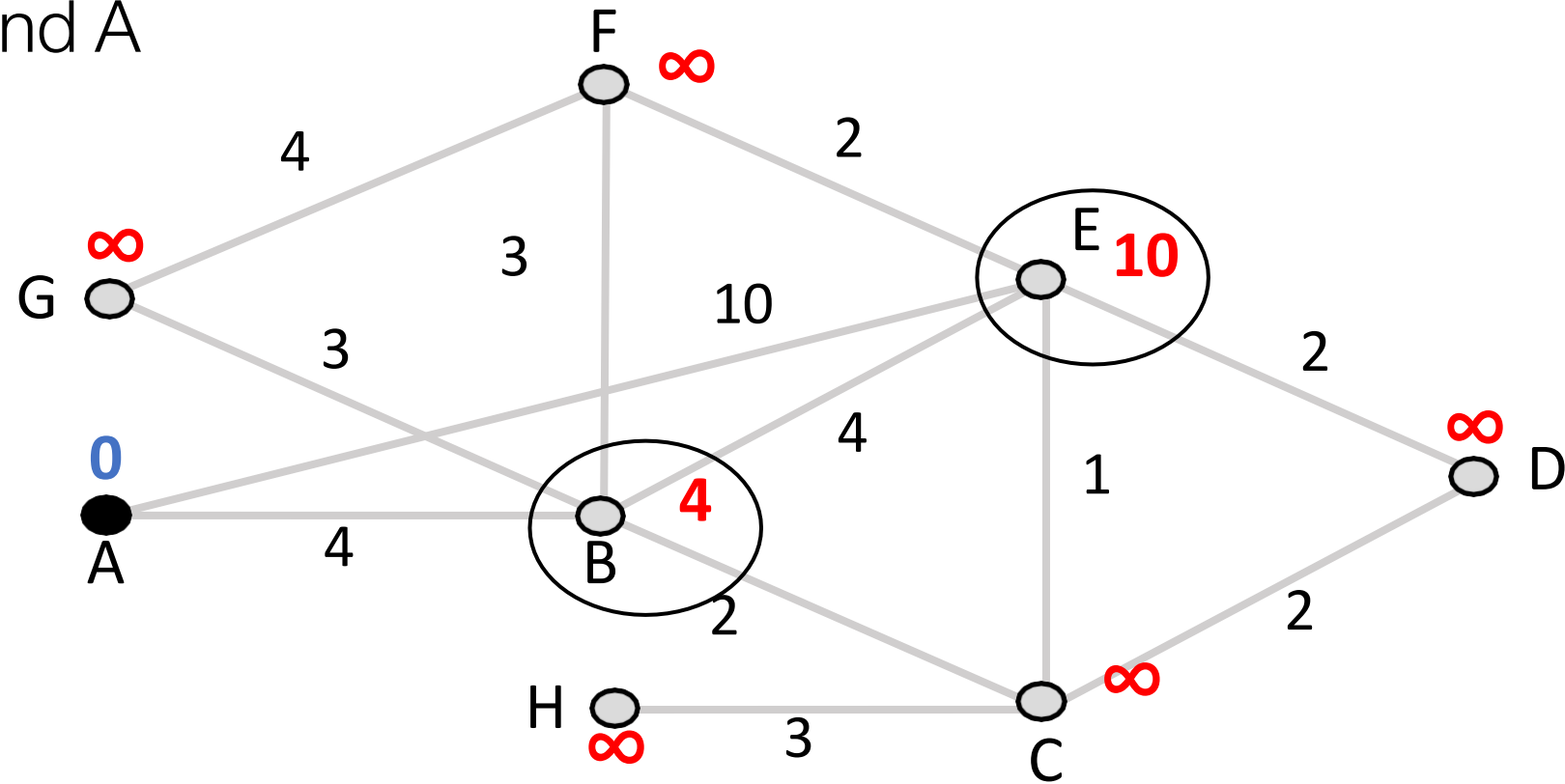
Dijkstra's Algorithm (2)

- Initialization



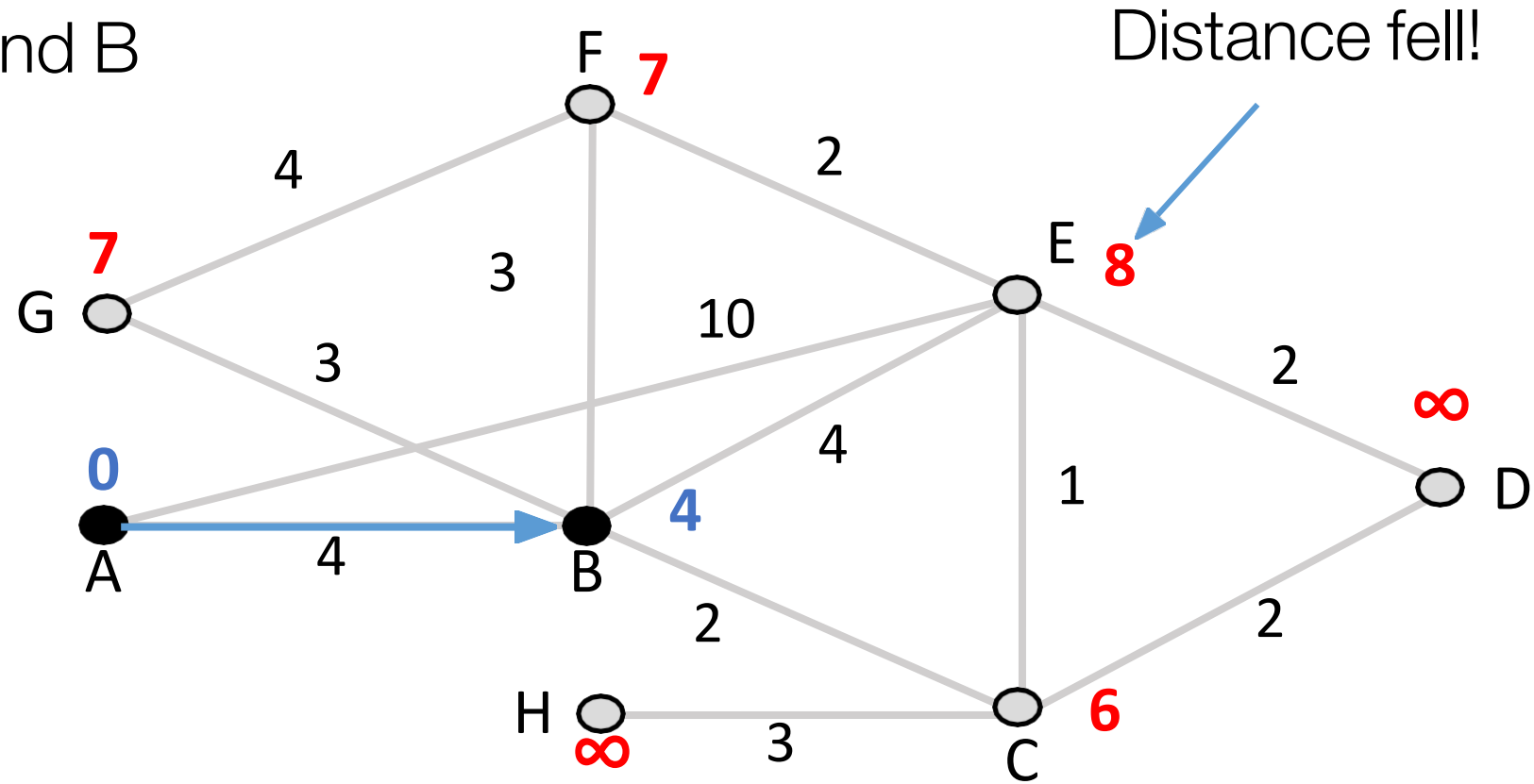
Dijkstra's Algorithm (3)

- Relax around A



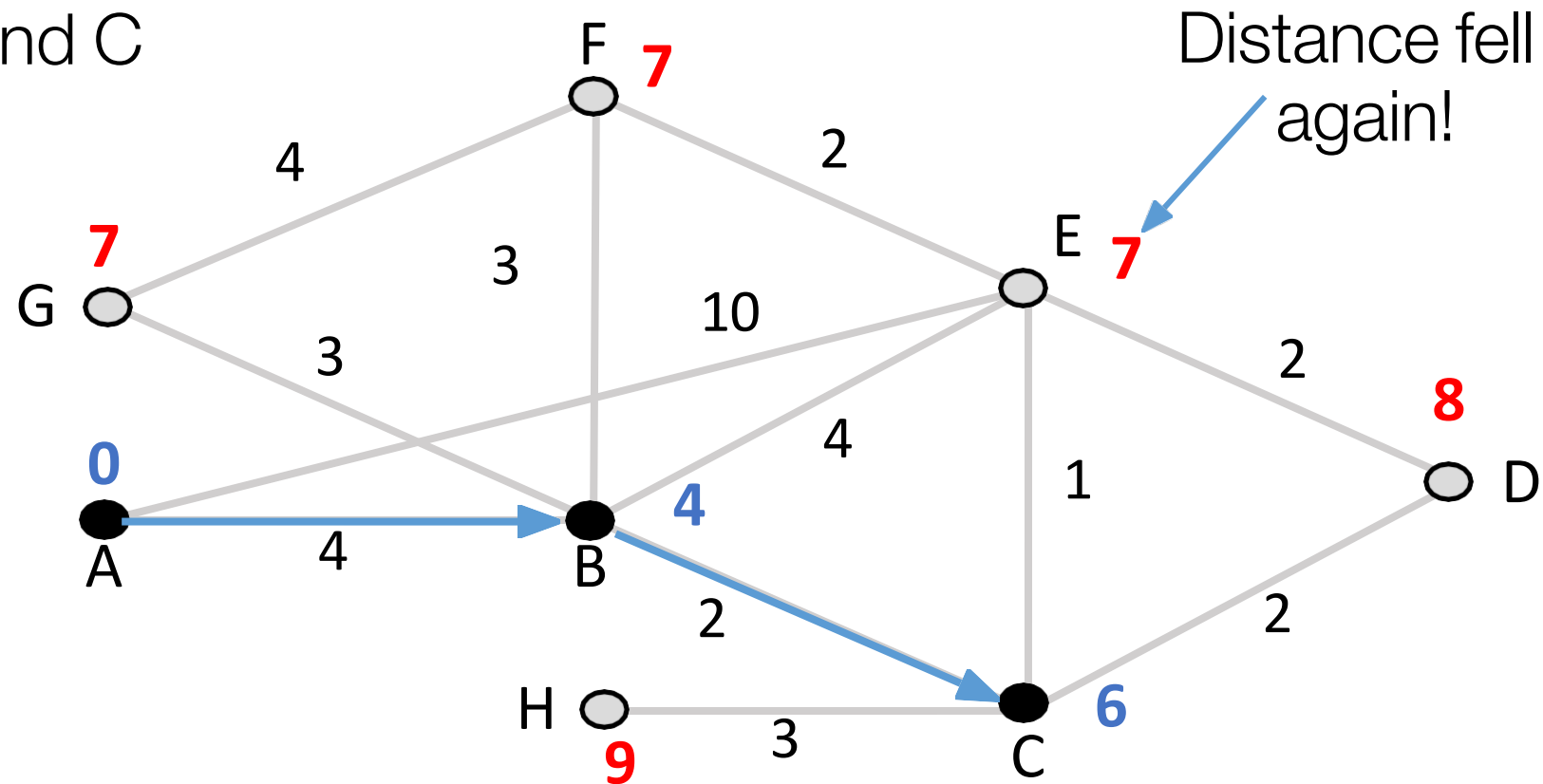
Dijkstra's Algorithm (4)

- Relax around B



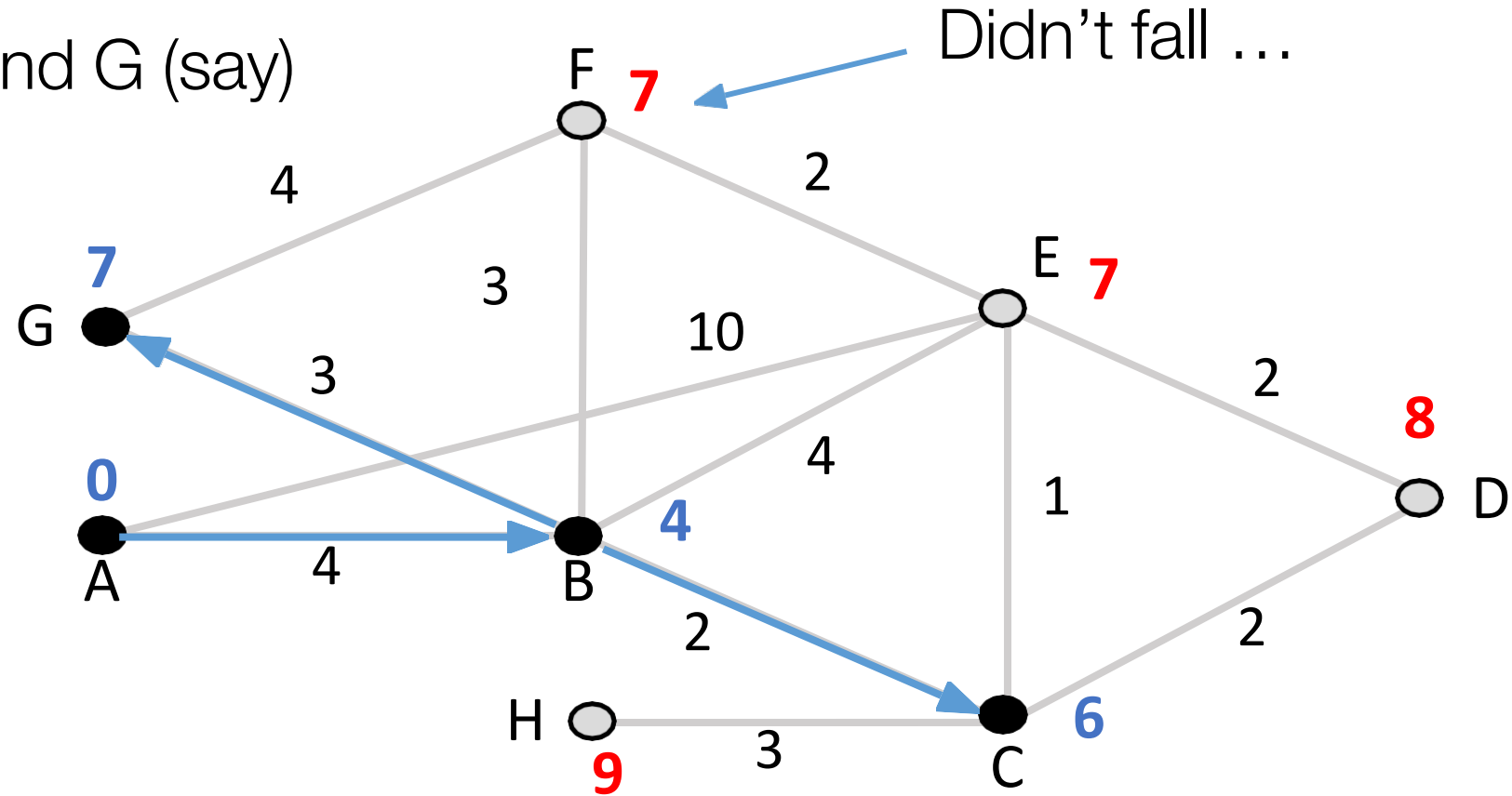
Dijkstra's Algorithm (5)

- Relax around C



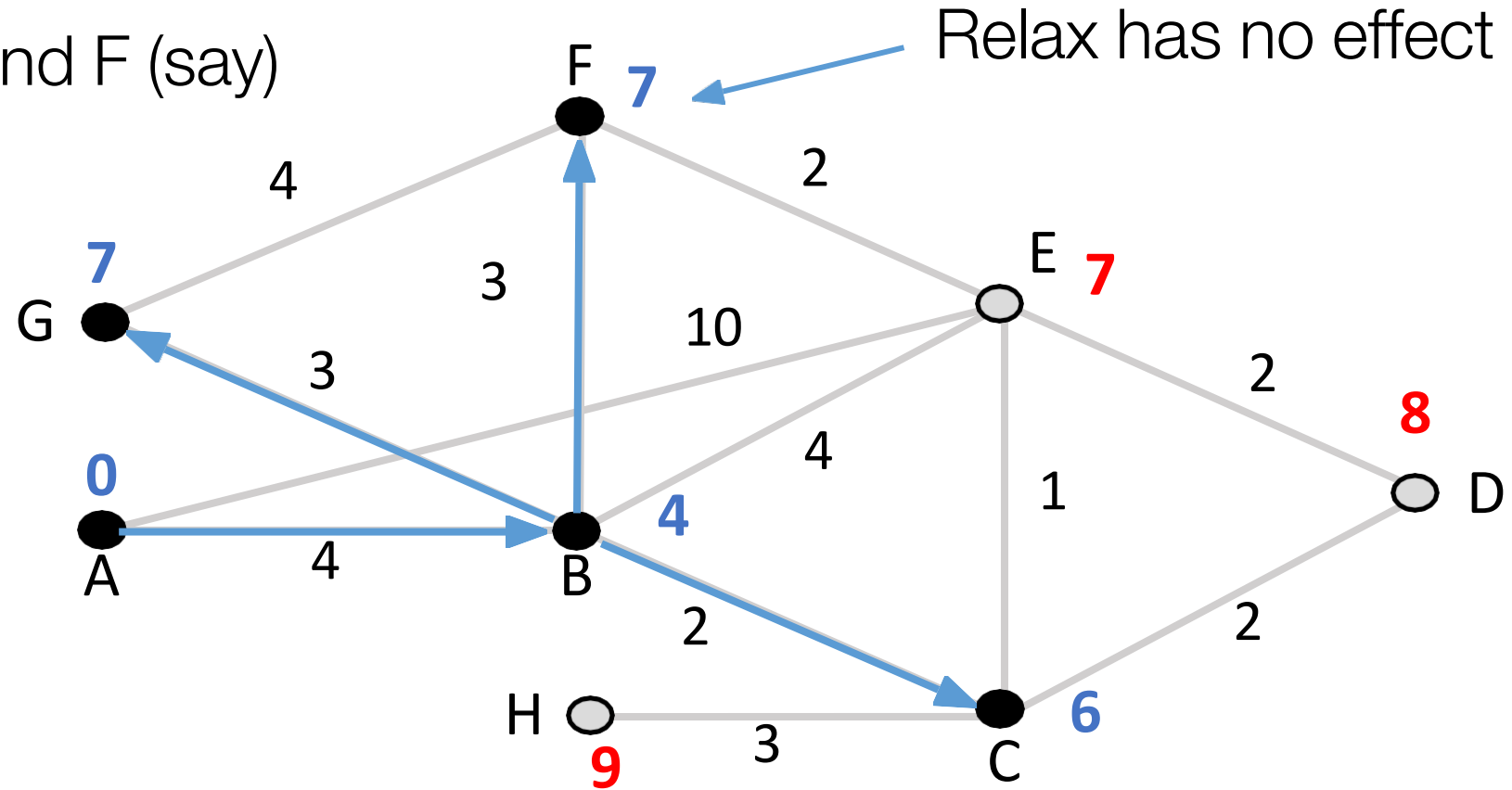
Dijkstra's Algorithm (6)

- Relax around G (say)



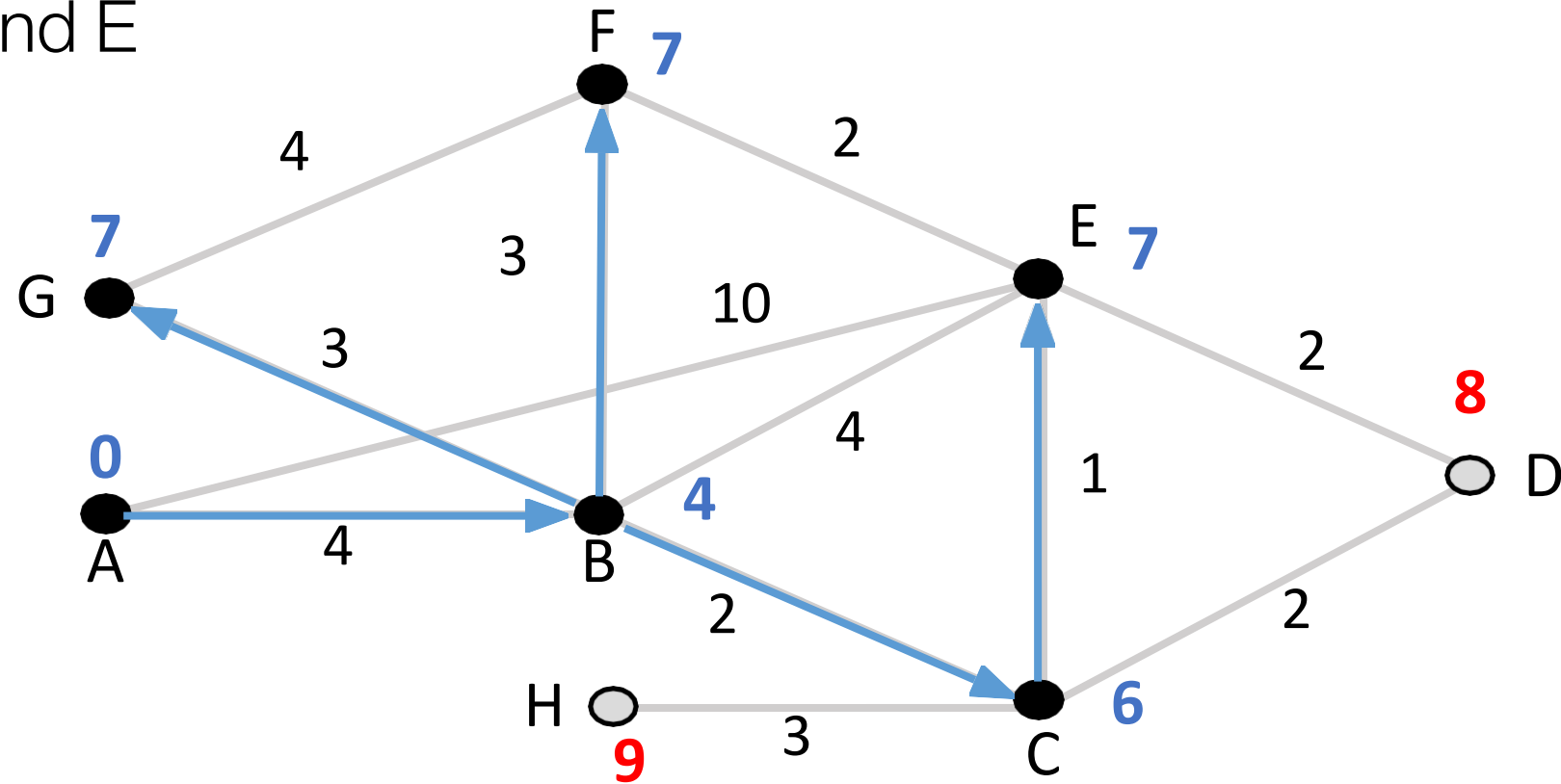
Dijkstra's Algorithm (7)

- Relax around F (say)



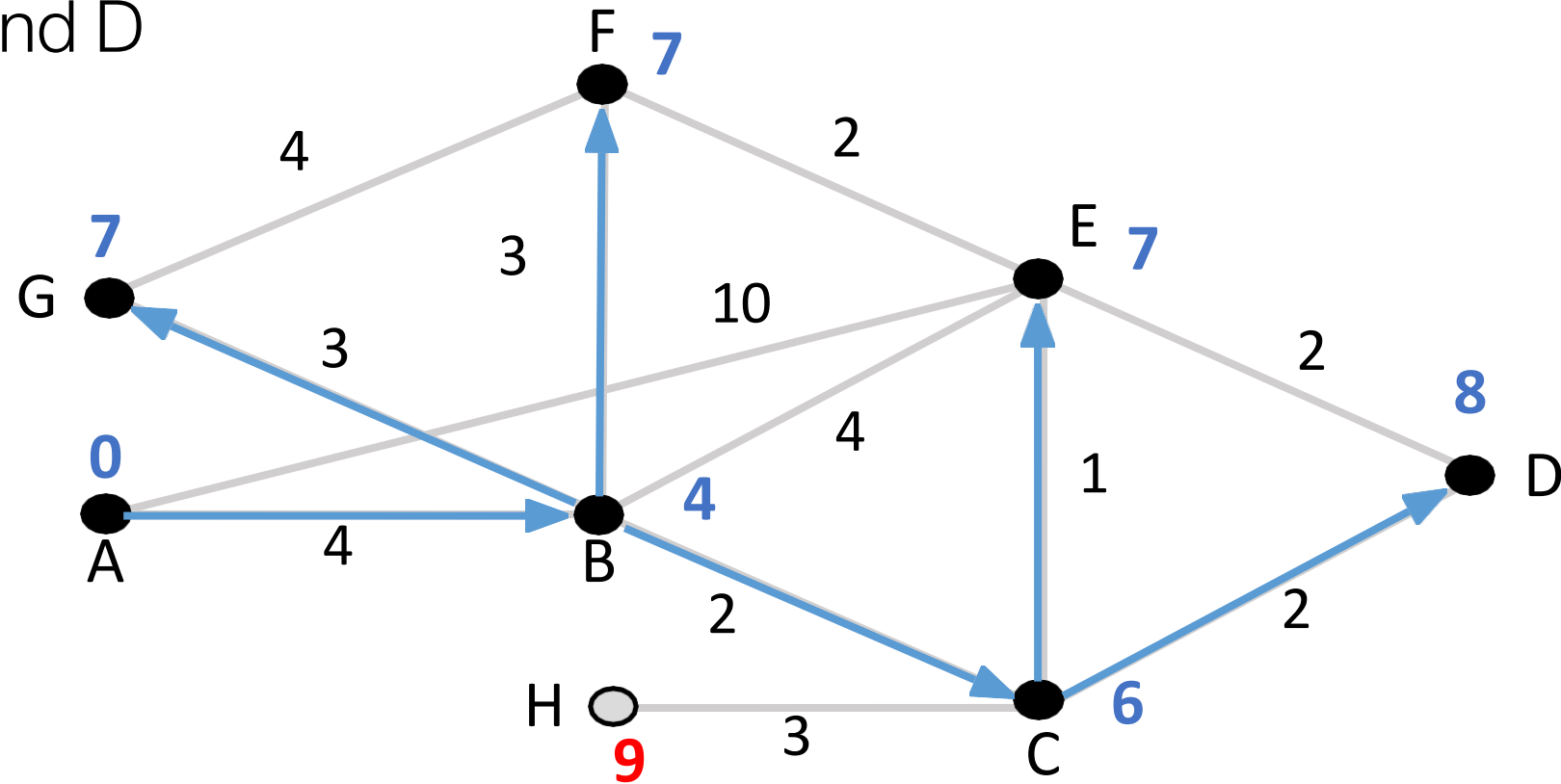
Dijkstra's Algorithm (8)

- Relax around E



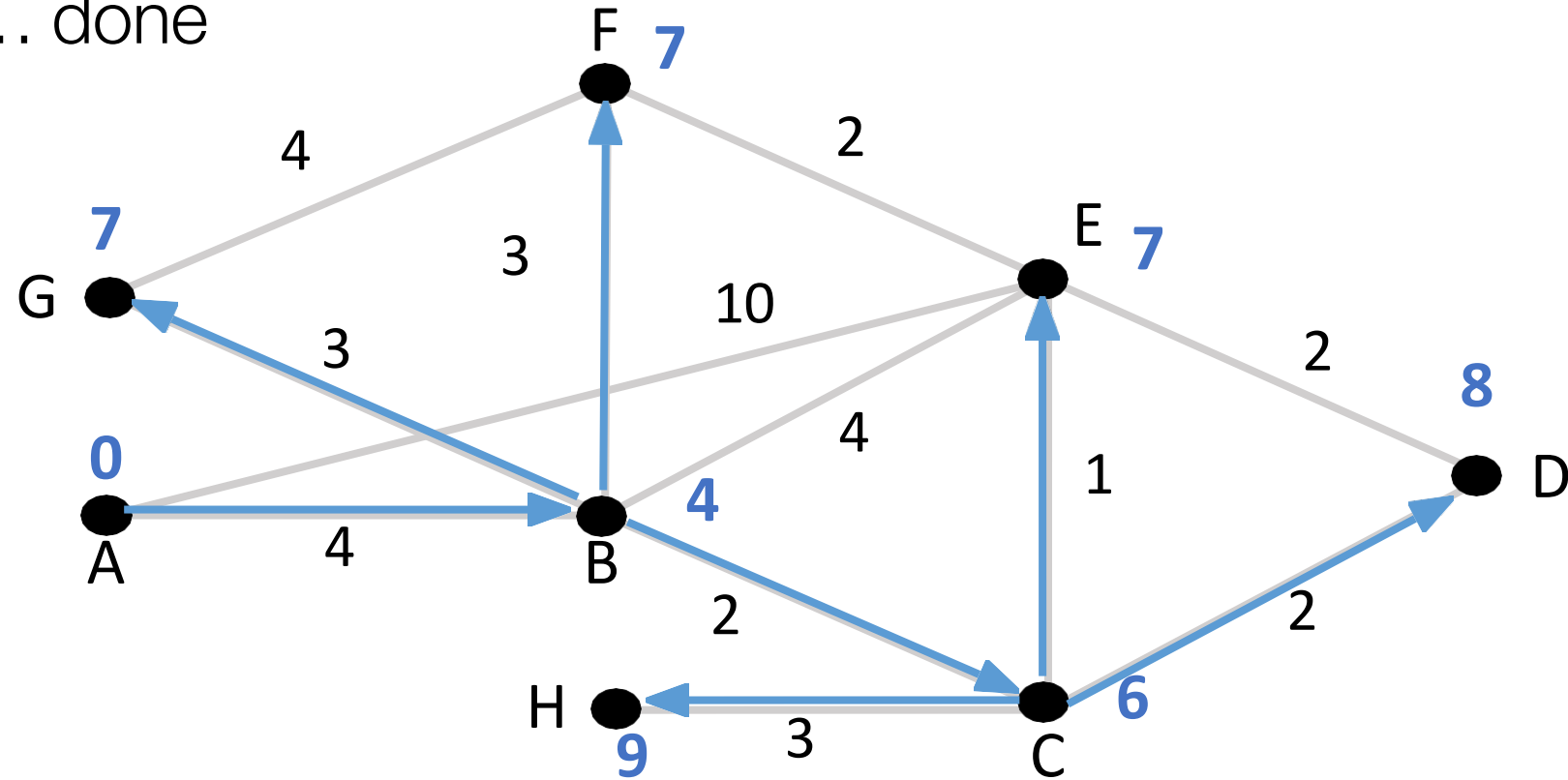
Dijkstra's Algorithm (9)

- Relax around D



Dijkstra's Algorithm (10)

- Finally, H ... done

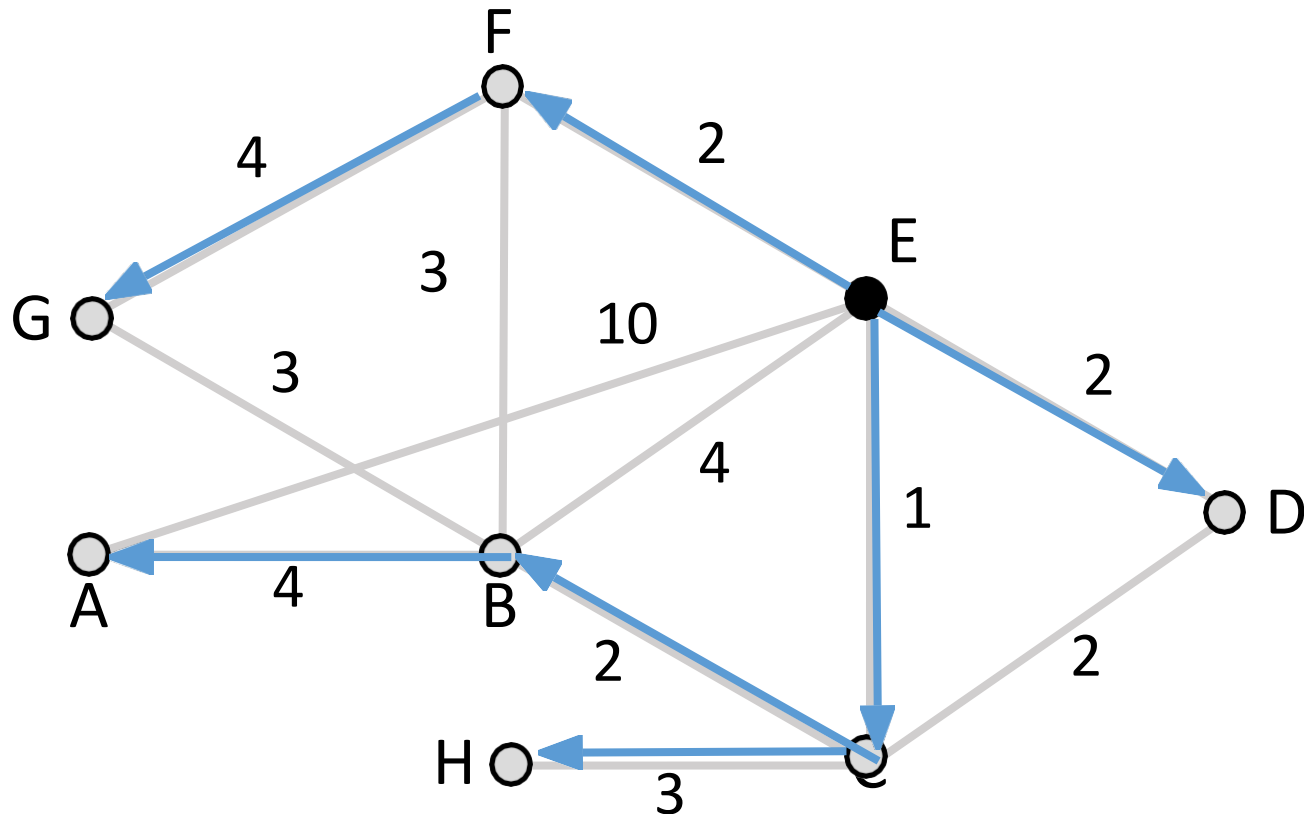


Dijkstra Comments

- Finds shortest paths in order of increasing distance from source
 - Leverages optimality property
 - Cost must be monotonic... no negative edges!
- Runtime depends on cost of extracting min-cost node
 - Superlinear in network size (grows fast)
 - Using binary heaps the complexity is $O((|E|+|V|)\log(|V|))$
 - $O(|E|+|V|\log(|V|))$ with Fibonacci heaps

Forwarding Table

Source Tree for E (from Dijkstra)



E's Forwarding Table

To	Next
A	C
B	C
C	C
D	D
E	--
F	F
G	F
H	C

Handling Changes

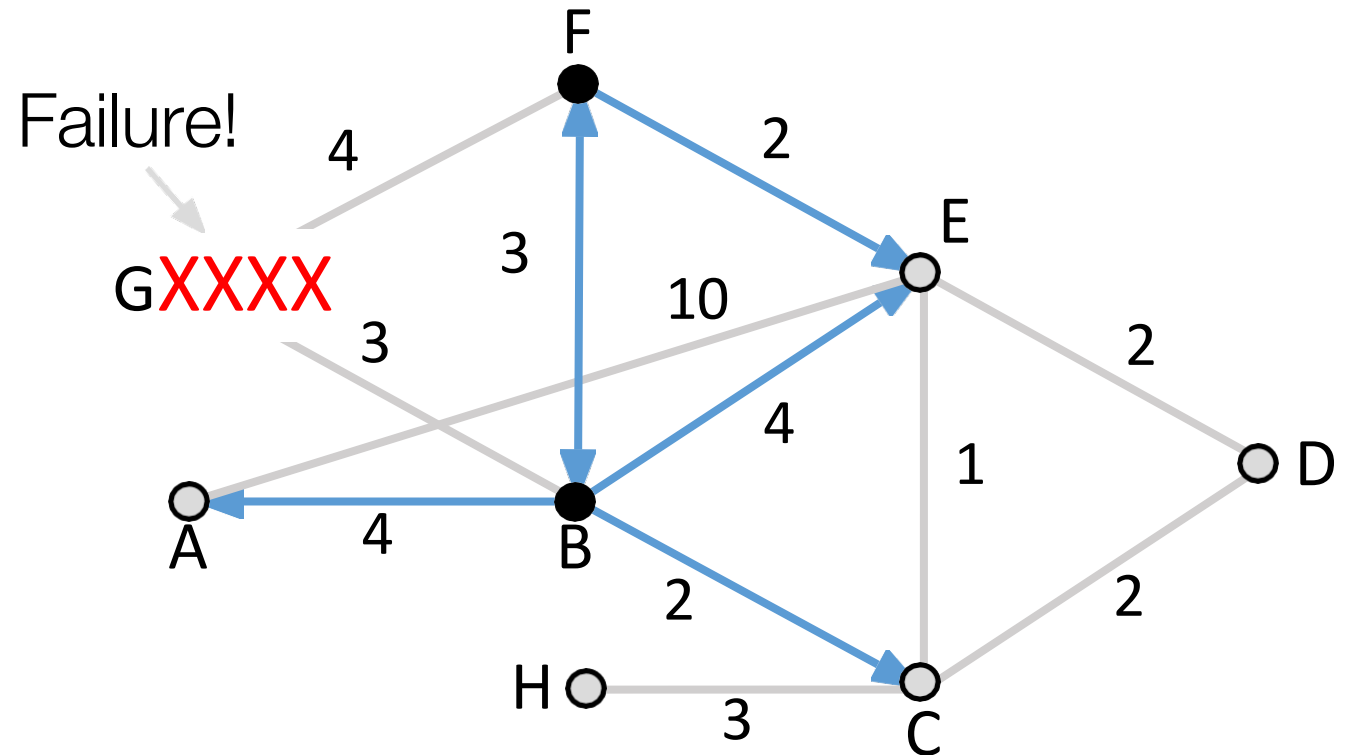
- On change, flood updated LSPs, re-compute routes
 - E.g., nodes adjacent to failed link or node initiate

B's LSP

Seq. #	
A	4
C	2
E	4
F	3
G	∞

F's LSP

Seq. #	
B	3
E	2
G	∞



Handling Changes (2)

- Link failure
 - Both nodes notice, send updated LSPs
 - Link is removed from topology
- Node failure
 - All neighbors notice a link has failed (link state!)
 - Failed node can't update its own LSP
 - But it is OK: all links to node removed

Handling Changes (3)

- Addition of a link or node
 - Add LSP of new node to topology
 - Old LSPs are updated with new link
- Additions are the easy case ...

Link-State Complications

- Things that can go wrong:
 - Seq. number reaches max, or is corrupted
 - Node crashes and loses seq. number
 - Network partitions then heals
- Strategy:
 - Include age on LSPs and forget old information that is not refreshed
- Much of the real-world implementation complexity is due to handling corner cases

Credits

- Some slides are adapted from course slides of CSE 461 in UW